

Verxeon Technologies

Legal Document Generator Case Study & Individual Intern Task Plan

A Full-Stack, Supabase-Powered Wizard for Drafting Lawyer-Reviewable Legal Documents

1. Case Background

Imagine you have been hired as a **Full-Stack Developer** by **Meridian Legal Group**, a law firm that wants a working document generator: a template library, a multi-step wizard form, and a document preview pane. Right now none of it exists there is no interface and no data behind it. It all needs to be built from the ground up.

Your responsibility is to build **every part of this generator**, front to back: the interface itself and a real **Supabase** backend behind it, wired together end to end. You will design the screens, build the wizard, and connect every screen to a live database. That is the whole scope. One document-generation product, one Supabase project, every feature built and made real.

The solution should be **reusable and replicable**. Once the generator works for Meridian Legal Group, the same schema, template engine, and SQL scripts should be redeployable for any firm by swapping the seed templates and the branding.

Business Problem:

Associates currently start every simple will, NDA, or lease agreement from a blank page or an outdated Word file buried in someone's inbox, retyping the same boilerplate clauses every time and occasionally missing a required clause entirely. Clients wait days for a first draft that should take minutes. The firm needs a system where a client or paralegal answers a short, guided form and a correctly formatted draft comes out the other end ready for an attorney to review, not to write from scratch. Every one of these is a missing piece an interface screen, a template, a dynamic field, a generation step, a review status that you are here to build, front to back.

Your Objective:

Design, build, test, and document a small production-quality full-stack application that delivers a working document generator: real dashboard metrics, a template library with dynamic fields, a working wizard that merges client answers into a template, a draft-review workflow for attorneys, client records, staff roles, and reports all built on top of a properly modelled Supabase database with saved SQL scripts.

What you will learn from this case:

- How a real document-automation product is decomposed into templates, dynamic form fields, and a merge/generation step.
- How to design and build a working interface from scratch a template library, a dynamic wizard, and a review screen.
- How to model a relational schema in Supabase (Postgres) and write the SQL that creates it.
- How to wire a UI to a database: auth, dynamic forms, document generation, and row-level security.
- How to keep every SQL script version-controlled in the codebase so the build is reproducible.

2. Solution Architecture

The build is deliberately focused: **one document-generation product, one Supabase backend**. The Supabase Postgres database is the single source of truth for every template, client, generated document, and review comment. Supabase Auth handles logins. Everything the interface shows the template library, the wizard, the draft preview, the review queue is read live from the database, and every action writes back to it.

Layer	Component
Interface	A template library, a dynamic multi-step wizard, and a draft review screen, built for the generator
Database	Supabase Postgres tables for templates, template fields, clients, generated documents, review comments, staff, roles; the source of truth
Auth	Supabase Auth staff login, sessions, and role checks
Template engine	Merges wizard field values into a template's placeholder text to produce the generated draft
SQL scripts	Every table, policy, and seed saved as .sql files committed in the repo
Data access	Queries and mutations connect each screen to its tables
Security	Row-level security policies enforce what each role can read and write

Document-journey flow: a staff member **logs in** through Supabase Auth → the **Dashboard** reads live totals and activity from the database → an admin **builds a template** (e.g. "Simple Will," "NDA"), defining its dynamic fields → a client or paralegal **registers a client** → they **fill the wizard** for a chosen template, answering only that template's fields → the system **generates a draft** by merging the answers into the template's text → the draft enters the **review queue**, where an attorney reads it, leaves comments, and **approves or requests changes** → once approved, the document is marked **finalized** → an admin manages **staff roles and permissions** → they open **reports** that aggregate the real data.

Each screen is driven by live queries against the Supabase tables, which act as the source of truth and the trigger for every metric and view exactly how a real document-automation backend is wired.

3. Subsystem Reference

3.1 Templates & clients (Dashboard, Templates, Clients)

This subsystem guarantees that the firm's templates and client records are real, structured, and reusable.

- **Live dashboard:** total templates, documents generated this month, drafts pending review, and recent activity every figure read from the database, never hard-coded.
- **Template management:** create, edit, and delete templates; write the template body with placeholder text; define the *dynamic fields* (label, type, required) that drive the wizard for that template.
- **Client management:** add, view, and update clients, each with a document history drawn from their linked generated documents.
- **Search & filters:** template and client lists query the database directly so results are always current.
- **Status tracking:** every generated document carries a status draft, under review, changes requested, finalized that drives the review workflow.

3.2 Generation & review (Wizard, Review Workflow, Staff, Reports)

This subsystem produces drafts, routes them through review, and reports on all of it.

- **Wizard:** render a multi-step form built from a chosen template's dynamic fields, validate required answers, and save the answers.
- **Document generation:** merge the saved field values into the template's placeholder text to produce the generated draft, stored against that client.
- **Review workflow:** attorneys open the review queue, read a draft, leave comments, and move it to approved or changes-requested; the client-facing status updates accordingly.
- **Staff, roles & permissions:** manage users, assign roles, and enforce permissions with row-level security.
- **Reports:** documents generated per month, most-used templates, average time in review, and an outstanding-review-queue report, each aggregated live from the tables.
- **Settings:** firm information, role management, and the logged-in user's profile.

4. Individual Delivery Plan

Every intern completes **this entire plan individually** you each build your own complete backend from scratch so you personally experience and master every part. Do not split the work between interns. Tasks are ordered by dependency **finish earlier tasks before starting later ones**.

Theme	Outcome
Setup & schema	Interface scaffolded, Supabase project live, database schema created via SQL scripts.
Templates & dashboard	Template CRUD with dynamic fields works; the dashboard shows live metrics.
Clients & wizard	Customer CRUD works; the wizard generates a real draft from a template.
Review & staff	The review workflow and staff roles/permissions are fully functional.
Reports, settings & demo	Reports aggregate live data, settings work, and a full demo is recorded.

4.1 Setup & Schema

T1.1 Scaffold the interface & set up Supabase *Objective:* a running interface connected to a live Supabase project.

- Scaffold a new frontend project (e.g. Next.js) and name your build (e.g. MeridianDocGen-yourname).
- Point your project at the team repository <https://github.com/verxeon-ai/DocGen.git> and create your own branch to work in (see Section 7 for the Git steps).
- Create a free Supabase account and a new project; save the project URL and keys in a local `.env`.
- Connect the interface to Supabase with the client SDK and confirm the app boots without errors.
- Build the sidebar and navigation to the generator's sections: Dashboard, Templates, Clients, Wizard, Review Queue, Staff, Reports, Settings.

Definition of Done: the interface runs, connects to Supabase, shows the navigation, and your work is committed on your own branch.

T1.2 Design & create the database schema *Objective:* a relational schema that models the whole document-generation flow, created from saved SQL.

- Design tables: `document_templates`, `template_fields`, `clients`, `generated_documents`, `review_comments`, `staff`, `roles`, and supporting link tables.
- Write the `CREATE TABLE` SQL with keys and foreign-key relationships (e.g. a generated document references a template and a client).
- **Save every SQL script** into a `/supabase/sql` folder in the repo so the schema is reproducible.
- Add a seed script that inserts sample templates (e.g. Simple Will, NDA), their fields, and sample clients and staff for testing.

Definition of Done: the schema exists in Supabase, all SQL scripts are committed to the repo, and seed data loads.

4.2 Templates & Dashboard

T2.1 Template management with dynamic fields *Objective:* the template library is fully manageable and drives the wizard.

- Wire *Add*, *Edit*, and *Delete Template* to the `document_templates` table.
- Build a *field builder* so an admin can define each template's dynamic fields (label, type, required) into `template_fields`.
- Write the template body with placeholder tokens (e.g. `{{client_name}}`) that map to the defined fields.
- Test the full cycle: create a template, add fields to it, edit it, and delete it.

Definition of Done: templates and their dynamic fields can be created, edited, and deleted against real data.

T2.2 Live dashboard metrics & charts *Objective:* every dashboard figure reflects the real database.

- Replace hard-coded cards with live queries: total templates, documents generated this month, drafts pending review.
- Build a recent-activity feed from the latest generated documents and review actions.
- Wire the charts to aggregated data (e.g. documents generated over time, by template).
- Test that generating a new document immediately changes the dashboard numbers.

Definition of Done: all dashboard metrics, activity, and charts read live from the database and update on data changes.

4.3 Clients & Wizard

T3.1 Client management *Objective:* clients are real records with a full document history.

- Wire *Add*, *View*, and *Update Client* to the `clients` table.
- Build the *document history* view by querying that client's linked generated documents.
- Add search over the client list as a database query.
- Test creating a client, updating them, and viewing their document history.

Definition of Done: clients can be created, viewed, and updated, and each shows an accurate document history.

T3.2 Wizard & document generation *Objective:* filling the wizard produces a real, correctly merged draft.

- Render the wizard's steps dynamically from the chosen template's `template_fields`, validating required answers before advancing.
- Save the submitted answers and *merge* them into the template's placeholder text to produce the `generated_documents` row.
- Attach the generated document to the chosen client with an initial status of *draft*.
- Test filling out the wizard for a template and confirm the generated draft correctly reflects every answer.

Definition of Done: the wizard generates a correctly merged draft, saved against a client, for any template.

4.4 Review & Staff

T4.1 Draft review workflow *Objective:* attorneys can review, comment on, and approve or reject drafts.

- Build the *review queue* listing every document with status *draft* or *changes requested*.
- Wire *Add Comment* to the `review_comments` table, linked to a generated document and the reviewing staff member.
- Wire *Approve* and *Request Changes* actions that update the document's status.
- Test moving a draft through comments, a change request, a resubmission, and final approval.

Definition of Done: drafts can be reviewed, commented on, and moved through status to finalized.

T4.2 Staff, roles & permissions *Objective:* the right people can do the right things, enforced by the database.

- Wire *Users* management to the `staff` table, backed by Supabase Auth.
- Define *Roles* (e.g. admin, attorney, paralegal) and assign them to users.
- Enforce *Permissions* with row-level security policies, saved as SQL scripts in the repo e.g. only attorneys may approve a draft.
- Test that a paralegal cannot approve a draft while an attorney can.

Definition of Done: users have roles, RLS policies enforce permissions, and all policy SQL is committed to the repo.

4.5 Reports, Settings & Demo

T5.1 Reports, settings & demo *Objective:* live reporting, working settings, and a polished final demo.

- Build reports that aggregate live data: documents generated per month, most-used templates, average time in review, and the outstanding-review queue.
- Wire *Settings*: firm information, role management, and the logged-in user's profile.
- Confirm every SQL script (schema, seed, policies) is saved in the repo and the database rebuilds from them.
- Record a 4–5 minute demo of the full document journey, from filling the wizard to a finalized, approved document.

Definition of Done: reports and settings work against live data, all SQL is committed, and the demo is recorded.

5. Individual Task Checklist

Each intern must personally complete and be able to demonstrate **all** of the following in their own interface, backed by **their own Supabase project**. Tick each item only when you can demo it working live.

#	Feature	You can demonstrate that...
1	Interface built & connected	the app runs and connects to a live Supabase project.
2	Database schema & SQL scripts	the schema is created and every SQL script is committed to the repo.
3	Live dashboard metrics	totals, activity, and charts read live from the database.
4	Template management	templates can be added, edited, and deleted against real data.
5	Dynamic template fields	each template's fields can be defined and drive its wizard.
6	Customer management	clients can be added, viewed, updated, with document history.
7	Wizard & generation	the wizard renders a template's fields and produces a correctly merged draft.
8	Review workflow	drafts can be commented on and moved through approve/changes-requested status.
9	Draft status tracking	every document's status (draft, under review, finalized) is accurate and visible.
10	Staff, roles & RLS	users have roles and permissions are enforced by row-level security.
11	Reports	documents per month, top templates, and review-queue metrics aggregate live data.
12	Settings	firm info, roles, and profile can all be updated.

6. Deliverables

Each intern submits the following individually at the end of the week:

- A live, running document generator connected to a working Supabase project with every page functional.
- Your work pushed to the team repository <https://github.com/verxeon-ai/DocGen.git>, developed on your own branch and merged into `main` only after testing.
- A repository containing the interface code **and all SQL scripts** (schema, seed, RLS policies) under `/supabase/sql`.
- One demo recording (4–5 minutes) of the full document journey, from wizard to finalized document.

7. Database & Repository Reference

Build the interface and the backend behind it. Keep every SQL script in the repository so the entire database can be rebuilt from scratch by anyone who clones it.

Item	Detail
Team repo (push here)	https://github.com/verxeon-ai/DocGen.git
Backend	Supabase (Postgres database and Auth)
SQL location	<code>/supabase/sql/</code> every script committed to the repo
Core tables (suggested)	
<code>document_templates</code>	name, category, template body with placeholder tokens, status
<code>template_fields</code>	template_id (FK), label, field type, required flag, order
<code>clients</code>	name, contact details, address, notes
<code>generated_documents</code>	template_id (FK), client_id (FK), field values, generated text, status
<code>review_comments</code>	generated_document_id (FK), staff_id (FK), comment, resolved
<code>staff</code>	linked to Supabase Auth user, name, contact
<code>roles</code>	role name and permission set, joined to staff
Required SQL scripts	
<code>schema.sql</code>	all CREATE TABLE statements with keys and foreign keys
<code>seed.sql</code>	sample templates, fields, clients, and staff for testing
<code>policies.sql</code>	row-level security policies for roles and permissions

8. Git Workflow (Simple Guide)

You will push all your work to the team repository <https://github.com/verxeon-ai/DocGen.git>. **Never build directly on main**. Always work on your own branch, test everything, and only then merge into `main`. Follow these steps in order they are all you need.

Step 1 Get the code onto your machine (once):

- `git clone https://github.com/verxeon-ai/DocGen.git`
- `cd DocGen`

Step 2 Create your own branch and switch to it:

- `git checkout -b yourname-docgen` (use your own name, e.g. `ali-docgen`).
- This keeps your work separate from `main` and from other interns.

Step 3 Work, then save your progress often:

- After finishing a piece of work: `git add .`
- Save it with a clear message: `git commit -m "Add template field builder"`
- Send it to GitHub: `git push origin yourname-docgen`
- Repeat this every time you complete a task small, frequent commits are best.

Step 4 Test before you merge:

- Run the app and click through every page you changed.
- Make sure nothing is broken and every feature works against Supabase.
- Only move on once you are confident your branch is fully working.

Step 5 Merge into `main` (only after testing):

- Open a Pull Request on GitHub from your branch into `main`, or ask your lead to review it.
- Once approved and tested, merge it into `main`.
- **Golden rule:** code only reaches `main` after it has been tested and works.

Handy tip: to update your branch with the latest `main` at any time, run `git checkout main`, then `git pull`, then `git checkout yourname-docgen`, then `git merge main`.